



Adventist University of Central Africa

P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

Department of Information Technology

Course Code and Name: INSY 8311 – Database Development with PL/SQL

Assignment 2: PL/SQL Fundamentals (Variables, Conditions and Loops)

Released Date: Sunday, 27th October 2026

Deadline: Sunday, 4th October 2026, 11:59 PM

1. Purpose

Assignment 1 checked your command of plain SQL. This assignment is your first step into PL/SQL: anonymous blocks, variables, %TYPE and %ROWTYPE, SELECT INTO, IF/ELSIF/ELSE, CASE, and the three loop types (basic LOOP, WHILE and FOR). It uses the same schema you already set up in your XEPDB1 database as the **lecturer** user: **STUDENTS**, **COURSES**, **ENROLLMENTS**, **EMPLOYEES**, and **DEPARTMENTS**.

Section A is a short SQL recap. PL/SQL does not replace SQL: every block you write in Sections B to F still depends on correct SELECT statements inside it, so SQL practice continues in every assignment. All other tasks must be solved with PL/SQL anonymous blocks that print their results with DBMS_OUTPUT.PUT_LINE.

2. Submission Rules: Read Before You Start (Git Workflow)

Follow this exactly. Any deviation results in disqualification of your submission.

1. **Clone** the class repository, do not fork it, do not create your own new repository for this assignment.
2. **Create your own branch** dedicated to this assignment. Use the naming convention: **student_id_firstname_lastname_assignment2** (e.g. 20657_elie_kayitare_assignment2)
3. Do all your work on **that branch only**.
4. **Commit** your work with clear, incremental commit messages (not one giant "final" commit).
5. **Push your branch** to the remote repository.
6. **Do NOT push to main** and do NOT open a pull request that merges into main yourself. Submission is your pushed branch, nothing else.
7. Submitting via your own repository, a zip file, email, or any channel other than a pushed branch on the class repo will **not be graded**.

Disqualifying actions: pushing to main, creating a personal/separate repository for this work, or submitting outside your dedicated branch.

What to push

A single, dedicated folder for this assignment inside your branch:

1. Assignment2 folder with a .sql file, a README.md and a pdf report. Both the .sql file and the pdf report should follow the same naming convention as your branch.

Each block in the .sql file must start with a comment header like the one below, followed by comments explaining the code you have used:

```
-- =====  
-- TASK B1: Variables and DBMS_OUTPUT  
-- Student: [Your Name] | ID: [Your ID]  
-- =====
```

The **pdf report** must contain every query and block you wrote together with a screenshot of its output. The **README.md** must describe the approach you used, the lessons you learned and the challenges you faced.

The README.md and pdf report must be written by you. They must reflect your own testing process for your own code. Copying another student's report, or generating it with an AI tool instead of describing your own verification steps, will be treated as an academic integrity violation.

3. Ground Rules

- ✓ Every query and block must compile and run without error against the schema from the install/config guide (50 students, 15 courses, 75 enrollments, 40 employees, 6 departments). Run SET SERVEROUTPUT ON before testing.
- ✓ Use the construct the task asks for. Where a task says "use a WHILE loop", "use a searched CASE" or "use %ROWTYPE", you must use that technique. Solving it a different way earns partial credit at best.
- ✓ Declare variables with %TYPE or %ROWTYPE whenever they hold a column value, instead of guessing the data type.
- ✓ Do not hardcode values you could otherwise derive (e.g. don't type the number of Computer Science employees; count it with SQL). The IDs given in a task statement may be used as given.
- ✓ Where a task shows a sample output, match its layout. The numbers must come from your own database, so they may differ from the sample.
- ✓ Code that does not compile receives 0 for that task. Comment your code: a correct block with zero explanation of intent loses style points.

4. Tasks (100 points)

Section A: SQL Recap (10 pts)

A1 (10 pts). This question keeps the SQL from Assignment 1 active. Write the two queries below as plain SQL (no PL/SQL).

- **(a) (6 pts)** For every department in DEPARTMENTS, show the department name, location, number of employees, total salary bill, average salary (rounded to 2 decimal places) and the number of courses the department offers. Match EMPLOYEES.department and COURSES.department to DEPARTMENTS.department_name. Departments with no employees or no courses must still appear, showing 0. Use a LEFT JOIN for the employees and a correlated subquery for the course count, and order the result by total salary bill, highest first.
- **(b) (4 pts)** Using GROUP BY and HAVING, list only the departments whose total salary bill is greater than their budget, showing department name, budget, total salary bill and the amount over budget.

In your README, explain in two or three sentences why joining EMPLOYEES and COURSES to DEPARTMENTS in the same query would give wrong counts. Keep the output of (a): you will use it to check your answer to Task E2.

Section B: Variables, %TYPE, %ROWTYPE and DBMS_OUTPUT (15 pts)

B1 (5 pts). Write an anonymous block that declares and initializes the following variables, then prints all four values in a formatted way:

- a VARCHAR2(100) variable holding the name of any course from the COURSES table;
- a NUMBER(1) variable holding a number of credits between 1 and 5;
- a DATE variable initialized to DATE '2024-09-01';
- a BOOLEAN variable that is TRUE if the credits are 3 or more and FALSE otherwise. DBMS_OUTPUT cannot print a BOOLEAN directly, so convert it to text before printing.

```
Course: Database Management Systems
Credits: 4
Semester Start: 01-SEP-24
Heavy Course: TRUE
```

B2 (5 pts). Using SELECT INTO and variables declared with %TYPE, fetch the course name, instructor, credits and max_students of course_id 110 (Machine Learning) and print them. If the course has 25 or fewer seats (max_students), also print 'Limited seats available!'.

B3 (5 pts). Declare a variable of type employees%ROWTYPE and fetch the full row for employee_id 22 (Amy, Full Professor) into it. Print her full name, department, salary and hire year (use EXTRACT(YEAR FROM hire_date)). Then calculate and print how many complete years she has worked at the university.

Section C: IF / ELSIF / ELSE Conditions (15 pts)

C1 (5 pts). Employee Commission Checker. Fetch employee_id 1 (John Manager). Using IF/ELSIF/ELSE, print exactly one of the following messages:

- commission_pct IS NOT NULL and salary > 80,000: 'High-value manager with commission'
- commission_pct IS NOT NULL and salary <= 80,000: 'Manager with commission, standard tier'
- commission_pct IS NULL and salary > 90,000: 'Senior employee, consider adding a commission incentive'
- otherwise: 'Standard employee profile'

Also print the employee's total potential earnings: salary + (salary * NVL(commission_pct, 0)).

C2 (5 pts). Course Availability Check. Fetch course_id 103 (PL/SQL Programming) and count the students enrolled in it using the ENROLLMENTS table. Then use IF/ELSIF to decide its availability:

- enrolled < max_students * 0.5: 'Open, plenty of seats available'
- enrolled between max_students * 0.5 and max_students * 0.8: 'Filling up, limited seats'
- enrolled above max_students * 0.8 but below max_students: 'Almost full, enroll soon'
- enrolled >= max_students: 'FULL, enrollment closed'

Print the course name, maximum capacity, current enrollment and availability status.

C3 (5 pts). Student Age Check. Fetch student_id 1020 (Tom Clark) and calculate his age in whole years with TRUNC(MONTHS_BETWEEN(SYSDATE, date_of_birth) / 12). Use IF/ELSIF/ELSE to print 'Minor student, parental consent required' if he is under 18, 'Traditional student age' if he is 18 to 25, and 'Mature student, eligible for the evening program' if he is over 25. In a separate IF, add the note 'First-year cohort (2023)' if his enrollment_date is in 2023, and 'Joined in [year]' otherwise.

Section D: CASE Statements and Expressions (15 pts)

D1 (5 pts). Score to Grade Conversion. Fetch the score and stored grade of enrollment_id 5. Use a searched CASE statement to convert the score to a letter grade: 90 and above 'A', 80 to 89.99 'B', 70 to 79.99 'C', 60 to 69.99 'D', below 60 'F', and 'Not yet graded' if the score is NULL. Compare your computed grade with the stored grade and print either 'Grade match: YES' or 'Grade match: NO (stored X, computed Y)'.

D2 (5 pts). Department Building Codes. Fetch the Computer Science row from DEPARTMENTS. Use a simple CASE on department_name to assign a building code: 'Computer Science' 'BLD-CS | ICT Wing', 'Mathematics' 'BLD-MT | Science Wing', 'Business' 'BLD-BS | Commerce Wing', 'Engineering' 'BLD-EN | Technical Wing', 'Psychology' 'BLD-PS | Humanities Wing', any other 'BLD-GN | General Wing'. Then use a CASE expression inside the assignment

or the PUT_LINE to print the budget status: above 600,000 'Well Funded', 400,000 to 600,000 'Adequately Funded', below 400,000 'Underfunded'.

D3 (5 pts). Job Title Rank. Fetch employee_id 14 (Nina, Assistant Professor). Use a searched CASE on job_title to assign a rank number and rank title:

- 'Full Professor': Rank 1, 'Senior Academic'
- 'Associate Professor': Rank 2, 'Mid Academic'
- 'Assistant Professor': Rank 3, 'Junior Academic'
- any title containing 'Developer' or 'Engineer': Rank 4, 'Technical Staff'
- any title containing 'Admin' or 'Coordinator': Rank 5, 'Administrative Staff'
- anything else: Rank 6, 'General Staff'

Print the employee name, job title, rank number, rank title and salary. Then use an IF to print 'Promotion eligible' if the rank is 2 or better and the salary is below 100,000, and 'Not eligible for promotion yet' otherwise.

Section E: Loops (25 pts)

E1 (8 pts). Basic LOOP: Tuition Fee Calculator. Assume tuition of RWF 150,000 per credit. Using a basic LOOP (LOOP ... EXIT WHEN ... END LOOP), add courses one at a time in course_id order starting from 101, fetching each course's name and credits from COURSES. Keep a running total of credits and tuition and exit when the total reaches 15 credits or more. For each iteration print a line like the one below, and after the loop print the final total and the average cost per credit.

```
Added: Introduction to Programming (3) | Total: 3 credits | RWF 450,000
```

E2 (8 pts). WHILE Loop: Department Salary Report. Using a WHILE loop, process every employee in the Computer Science department (you may load them into a collection first, or use an explicit cursor and loop while rows are found). For each employee, add to the total salary, count high earners (salary above 75,000) and low earners (salary below 65,000), and track the highest-paid employee. After the loop, print the report below. Your totals must agree with the Computer Science row of your answer to A1(a).

```
===== Computer Science Salary Report =====
Total Employees: X
Total Salary Bill: RWF X
Average Salary: RWF X
High Earners (>75k): X
Low Earners (<65k): X
Highest Paid: [Name] - RWF X
=====
```

E3 (9 pts). FOR Loop: Academic Performance Table. Using a numeric FOR loop over student IDs 1001 to 1010, fetch each student's name and GPA and count their enrollments. Use a CASE to assign a performance label (GPA 3.9 and above 'Summa Cum Laude', 3.7 to 3.89 'Magna Cum

Laude', 3.5 to 3.69 'Cum Laude', 3.0 to 3.49 'Satisfactory', below 3.0 'Probation') and a second CASE for the course load (0 to 1 enrollments 'Under-enrolled', 2 to 3 'Normal Load', 4 or more 'Full Load'). Print one formatted row per student, then the number of students printed and the count per performance label.

ID:1001	Alice Johnson	GPA:3.8	Magna Cum Laude	Full Load
ID:1002	Bob Smith	GPA:3.2	Satisfactory	Normal Load

Section F: Combined Challenge (20 pts)

F1 (20 pts). Write a single anonymous block that produces an employee profile report for employee_id 5 (Tom Engineer). The block must combine the concepts from Sections B to E as follows:

- **Step 1, Profile (5 pts):** fetch the full row with %ROWTYPE, calculate years of service with MONTHS_BETWEEN(SYSDATE, hire_date) / 12, and use IF/ELSIF/ELSE to classify the service: under 2 years 'Probationary', 2 to 5 years 'Confirmed', over 5 years 'Senior Staff'.
- **Step 2, Salary Analysis (5 pts):** use a CASE to assign a pay grade from 1 to 5 using salary ranges you design (state them in a comment). If the service is over 3 years and the salary is below 85,000, print 'Raise recommended' and a proposed salary of current salary * 1.10; otherwise print 'No raise due'.
- **Step 3, Department Summary (6 pts):** use a FOR loop over all employees in Tom's department to count his colleagues (excluding Tom), find the highest salary in the department, and count how many colleagues earn more than Tom.
- **Step 4, Report (4 pts):** print the report in the layout below, with salaries formatted with thousands separators (e.g. TO_CHAR(salary, '999,999')).

```
=====
EMPLOYEE PROFILE REPORT
AUCA Staff Records System
=====
Employee: Tom Engineer (ID: 5)
Department: Computer Science
Job Title: Software Engineer
Hire Date: 15-SEP-19 | Years of Service: X.X
Service Status: X
-----
Salary: RWF 80,000 | Pay Grade: X
Raise Status: X
Proposed Salary: RWF X
-----
Dept. Colleagues: X
Dept. Highest Salary: RWF X
Colleagues Earning More: X
=====
```

Section G: SQL(5 pts)

G1 (5 pts). Write one plain SQL query (no PL/SQL) that returns the same figures as your E2 report: total employees, total salary bill, average salary, number of high earners, number of low earners, and the name and salary of the highest-paid employee in Computer Science. Put the SQL

output and your E2 output side by side in your pdf report and comment on which approach you would use in practice, and why.

A perfect answer submitted outside the correct branch, or pushed to main, receives **0** for the assignment regardless of code quality. **Late submissions if no extension was given lose 10 points per day for three days. If an extension was given, late submission will not be accepted.**

5. Academic Integrity

You may discuss approaches with classmates, but the SQL and PL/SQL you submit must be your own work, written and tested by you against your own database instance. Identical, unexplainable code files across two students will be treated as an academic integrity violation for both parties.

This includes the README.md and pdf report. They must be written in your own words, describing what you actually did and observed. A report that is copied, templated, or produced by an AI tool rather than reflecting your own verification work will be treated the same as copied code: an academic integrity violation.